

CSCE 645 GEOMETRIC MODELING · MODULE 12 · LECTURE NOTES

Solid Modeling and Booleans

Where exact geometry meets floating-point arithmetic.

MODULE OUTCOMES

- Explain boundary representation and its validity conditions.
- Describe the mesh boolean pipeline and where it fails.
- Explain robustness predicates and exact arithmetic.
- Explain why implicit and voxel approaches trade accuracy for robustness.
- Choose an approach from the required guarantees.

1 Boundary representation

A **B-rep** describes a solid by the surface that bounds it — the mesh is not the object, it is the object's skin. For that to define a solid at all, the surface must satisfy four conditions.

Condition	Meaning	Cost to check
Closed	No boundary edges; the surface has no holes.	Cheap: count half-edges with no twin.
Manifold	Module 01's two conditions on edges and vertices.	Cheap: local checks.
Consistently oriented	All face normals point outward.	Cheap: breadth-first traversal across shared edges.
Non-self-intersecting	No two faces pass through each other.	Expensive: requires all-pairs intersection testing, accelerated by a BVH. This is the condition real assets violate constantly.

Only when all four hold is 'inside' a well-defined notion, and only then do boolean operations mean anything.

1.1 Deciding inside

- **Ray parity.** Cast a ray from the query point and count surface crossings; an odd count means inside. Simple, and fragile: if the ray passes exactly through an edge or vertex, the count is ambiguous. Perturbing the ray direction helps and does not fully fix it.
- **Winding number.** Sum the signed solid angles subtended by all faces, divided by 4π . For a clean closed mesh this is 1 inside and 0 outside. It is numerically robust and degrades gracefully rather than catastrophically.

THE GENERALISED WINDING NUMBER

Jacobson and colleagues observed that the winding number remains meaningful on meshes that are *not* watertight: it becomes a smooth real-valued function that is close to 1 well inside the shape, close to 0 well outside, and intermediate near holes.

Thresholding it gives a sensible inside/outside classification for meshes with holes, self-intersections, and inconsistent orientation — which describes most meshes that arrive from a scanner or from another studio.

This has become the standard way to make downstream operations work on imperfect input, rather than demanding that the input be repaired first.

2 The boolean pipeline

- 1 **Find intersecting triangle pairs** between A and B, using a BVH to avoid the quadratic all-pairs test. Routine.
- 2 **Compute the intersection segments** for each pair and link them into closed intersection curves. **This is where it breaks.**
- 3 **Re-triangulate** both surfaces so that the intersection curves appear as mesh edges, using a constrained triangulation. Routine, given correct input from step 2.
- 4 **Classify and assemble:** each resulting patch is inside or outside the other solid, and the operation determines which patches to keep and whether to flip them. Routine.

DEGENERACIES ARE THE COMMON CASE

Real models are built with aligned geometry: faces that are exactly coplanar, edges that are exactly collinear, vertices that coincide precisely. Designers do this deliberately, because that is how parts fit together.

So step 2 is constantly asking questions like 'are these two planes the same plane?' and 'does this vertex lie exactly on that edge?', and floating-point arithmetic cannot answer them reliably. An epsilon too small misses genuine coincidences and emits degenerate slivers; too large merges features that should remain distinct. There is no correct epsilon.

The serious failure is not inaccuracy but **inconsistency**. If the code concludes that point p is above plane Q in one test and below it in a related test, the resulting combinatorial structure is impossible: a surface that does not close, or an edge with three incident faces. The output is not slightly wrong, it is invalid.

3 Buying robustness

Approach	What it does	What it costs
Exact predicates	Compute the <i>signs</i> of geometric predicates exactly, using floating-point filters that escalate to higher precision only when needed.	Modest slowdown. Does not give exact coordinates — only exact decisions, which is what matters.
Exact arithmetic throughout	Represent all coordinates as rationals.	Much slower, and coordinate representations grow without bound through chained operations.
Snap rounding	Round all coordinates to a fixed grid so near-coincidences become exact coincidences.	Alters the geometry, can collapse small features, and can still produce invalid topology.
Voxelise	Convert both solids to a volumetric grid, perform the boolean per voxel, extract a mesh.	Never fails. Rounds every sharp feature to the voxel size and produces a dense mesh needing remeshing.
Convert to implicit	Build distance fields and use min/max (Module 11).	Trivial and robust. The surface is approximate, and you gain smooth blending as a bonus.

EXACT PREDICATES ARE THE REAL ANSWER

The key observation is that a boolean algorithm does not need exact coordinates. It needs the correct **sign** of a small set of predicates: orientation of a point relative to a plane, whether four points are coplanar, in-circle and in-sphere tests.

Shewchuk's *adaptive precision* predicates evaluate those signs exactly. They begin with an ordinary floating-point computation plus a rigorous error bound; if the bound proves the sign, they return immediately at full speed. Only when the value is too close to zero do they escalate to extended precision.

In practice almost every call takes the fast path, so the cost is small, and every decision is correct. Correct decisions mean a *consistent* combinatorial structure, which is exactly what prevents impossible output.

CGAL, libigl, and every serious geometry kernel use them. The code is freely available and should never be rewritten by hand.

4 The voxel escape hatch

Converting to a volumetric or implicit representation makes booleans trivial, because inside/outside is a per-sample question with no combinatorial structure to corrupt. Union is a minimum, intersection a maximum, difference a negation — and no configuration of input geometry can cause a failure.

The cost is sharpness: every feature is rounded to the sample spacing, and marching cubes rounds corners further (Module 11). The output is also dense and usually needs remeshing.

THIS IS A LEGITIMATE PRODUCTION CHOICE, NOT A COP-OUT

3D printing pipelines routinely voxelise before booleaning, because a corner rounded by 0.1 mm is vastly preferable to a boolean that fails and produces a non-watertight model that cannot be sliced at all.

Game and VFX pipelines do the same for destruction, fluid interaction, and any case where robustness matters more than exact edges.

The question is never 'which approach is best' but 'what does this application actually require'. If the answer is manufacturing tolerance, use a CAD kernel with exact predicates. If it is reliability on arbitrary input, voxelise.

5 Choosing

Requirement	Approach
Exact results to manufacturing tolerance.	A NURBS B-rep kernel with exact predicates. This is what CAD software is.
Reliable results on messy, scanned, or third-party input.	Voxelise, or convert to a distance field. Accept the rounding.
Smooth blends rather than sharp intersections.	Implicit representation with <code>smin</code> (Module 11).
Interactive editing with immediate feedback.	Implicit, evaluated at render time — no mesh is extracted until export.
A game asset, booleaned once.	Do it offline in the modelling package, fix the result by hand, ship the mesh. This is the most common real answer and there is nothing wrong with it.

THE PRACTICAL SUMMARY

Robust mesh booleans have been an active research problem for four decades, and commercial kernels representing enormous investment still fail on adversarial input.

What is worth knowing: **why** it is hard (exact decisions from inexact arithmetic, with degeneracies as the common case), **what the real fix is** (exact predicates, giving consistency rather than accuracy), and **when to sidestep it** (voxelise when robustness beats sharpness).

Do not write your own. Use CGAL, libigl's `mesh_boolean`, or Cork, and understand what they are protecting you from.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Shewchuk — Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates (free, with code)	The paper and the freely usable implementation. The single most important reference for this module.
Jacobson et al. — Robust Inside-Outside Segmentation using Generalized Winding Numbers (free)	How to get meaningful inside/outside answers from broken meshes.
CGAL — Polygon Mesh Processing and Boolean operations	A production-quality implementation built on exact predicates. Read the preconditions section carefully.
libigl — mesh boolean tutorial	A usable boolean implementation with a short interface, built on CGAL/exact arithmetic.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement a B-rep validator checking all four conditions of §1. Run it on several downloaded models and report which conditions fail.
- 2 Implement ray-parity and winding-number inside tests. Construct a query point whose ray passes exactly through a vertex and show the parity test failing while the winding number succeeds.
- 3 Implement the generalised winding number on a mesh with a hole, and visualise it as a scalar field on a slice plane.
- 4 Attempt a naive mesh boolean on two cubes sharing a face. Document the exact point at which floating-point ambiguity defeats it.
- 5 Use Shewchuk's predicates for the orientation test in that code and report what changes.
- 6 Implement a voxel boolean: convert two meshes to occupancy grids, combine, and extract with marching cubes. Measure the feature rounding against voxel size.
- 7 Compare the voxel result against libigl's mesh_boolean on the same input, for both quality and robustness across twenty randomly perturbed configurations.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What four conditions must a mesh satisfy to represent a solid, and which is expensive to verify?
- 2 Why is the winding number more robust than ray parity, and what does the generalised version add?
- 3 Name the four steps of a mesh boolean and say which one fails.

- 4 Why are degeneracies the common case rather than an edge case?
- 5 What specifically goes wrong when a geometric predicate is answered inconsistently?
- 6 What do exact predicates compute exactly, and why is that sufficient?
- 7 Give a situation where voxelising is the correct engineering choice.